

DL-GenAI Project Report – T1 2026

(23f2002974)

1. Executive Summary

This project was developed for the **Messy Mashup competition**, which challenges models to classify music genres under extreme distribution shifts and noisy mixing conditions. The objective was to train a system using isolated instrument stems (drums, vocals, bass, and others) capable of accurately identifying **ten distinct music genres** from complex mashups that include rhythmic synchronization, tempo variations, and environmental noise. To address the gap between clean training data and the noisy test distribution, a robust audio processing pipeline was implemented. The approach involved converting audio signals into **Mel spectrogram representations** and introducing noise from the **ESC-50 dataset** to simulate real-world noisy conditions and improve model generalization.

Multiple deep learning architectures were evaluated, including a **CNN trained from scratch**, a **Recurrent CNN (RCNN)**, and the **Audio Spectrogram Transformer (AST)**. The models were trained to learn genre-specific patterns from the spectrogram representations. Performance was evaluated using the **Macro F1 Score**, ensuring balanced evaluation across all ten genre classes. Among the tested architectures, the **scratch CNN** achieved the best performance with a leaderboard Macro F1 score of **0.948**, followed by the **AST model** with **0.93**, while the **RCNN** achieved **0.802**. These results highlight that well-designed convolutional architectures can effectively capture local time–frequency patterns in music spectrograms, while transformer-based models also demonstrate strong capability in modeling global audio dependencies.

2. Introduction

The objective of the **Messy Mashup competition** is to classify **ten music genres** from complex audio mashups containing mixed instrument stems and environmental noise. The dataset includes clean music stems and noise samples, creating a challenging task due to distribution shifts between training and test data. The goal of this project was to build and evaluate deep learning models capable of robust genre classification under noisy conditions. Multiple architectures including **Scratch CNN, RCNN, and Audio Spectrogram Transformer (AST)** were implemented using **PyTorch**. The following sections describe the dataset, preprocessing pipeline, model architectures, training methodology, and evaluation results.

3. Dataset & Preprocessing

Dataset Description

The competition uses the **Messy Mashup dataset**, which contains clean music stems, environmental noise samples, and noisy mashups. The **genres_stems** folder provides clean

instrument stems from different music genres used for training. The **ESC-50 dataset** contains environmental sounds such as rain, animals, and mechanical noises used as noise sources. The **mashups** folder contains mixtures of music and noise used for evaluation. All audio signals are converted into **Mel spectrograms**, which provide a time–frequency representation suitable for deep learning models.

Exploratory Data Analysis (EDA)

Initial analysis was conducted to understand the structure and characteristics of the audio dataset. The dataset contains **10 music genres**, with **100 songs per genre**, resulting in a balanced dataset of **1000 songs**. Each song includes **four stems: drums, bass, vocals, and other**, and validation checks confirmed that all songs contain the required stems with **no zero-byte files**, indicating good data integrity.

The **average duration** of all stems is **approximately 30 seconds**, and bass, drums, vocals, and other stems have identical durations within each song. This confirms that the stems were correctly split from the same original track and are temporally aligned. Analysis of peak amplitude values showed that the **average vocal peak amplitude** is around **-12.49 dB**, while other stems exhibit different amplitude distributions, with drums generally having stronger peaks and bass slightly lower amplitude levels.

All **training** and **noise audio** files have a **sampling rate of 44,100 Hz**, while the **test dataset** uses **22,050 Hz**, requiring careful handling during preprocessing. Additionally, environmental **noise samples** are approximately **5 seconds long**, which were later used for augmentation by mixing them with music stems. Overall, the dataset is well balanced, consistently trimmed, and structurally clean, making it suitable for training deep learning models for genre classification.

Data Preprocessing

Audio files were first converted into **Mel spectrograms and saved as .npy files** before training. This preprocessing step avoids on-the-fly spectrogram computation during training and significantly **speeds up data loading and training time**.

Two input sizes were used for different models:

- A **CNN model trained from scratch** using audio segments of **50,000 samples**.
- An **AST (Audio Spectrogram Transformer)** model trained using **20,000 samples**.
- A **RCNN model trained from scratch** using audio segments of **50,000 samples**.

The saved spectrogram were then formatted into the appropriate shape for model input, and genre labels were encoded into numerical indices.

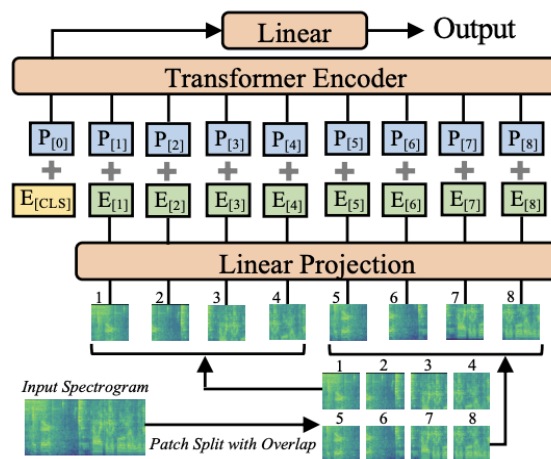
Data Augmentation

To improve generalization, several augmentation techniques were applied. Environmental noises from the **ESC-50 dataset** were randomly mixed with clean music stems to simulate noisy conditions. Additionally, **time masking** and **frequency masking** were applied to the

Mel spectrograms. These techniques randomly hide portions of the time or frequency axis, forcing the model to learn more robust audio features and reducing overfitting.

5. Modeling & Experimentation

5.1. Model 1: Fine-Tuned Audio Spectrogram Transformer (AST)



- **Architecture**

For this project, we used the **Audio Spectrogram Transformer (AST)** with the pretrained checkpoint **MIT/ast-finetuned-audioset**. AST is a **transformer-based architecture adapted from the Vision Transformer (ViT)** and designed specifically for audio classification tasks.

The model takes **log-Mel spectrograms as input** and divides them into small **overlapping patches**, which are then embedded into a sequence of tokens with positional encodings. These tokens are processed through **transformer encoder layers with self-attention**, allowing the model to learn relationships across the entire audio spectrogram. A final classification head predicts the **10 music genre classes**.

- **Salient Points**

Global Self-Attention: Unlike CNNs that focus on local patterns, AST can capture **long-range dependencies across the entire spectrogram**, which helps in learning genre-level musical structures.

Contextual Understanding: The transformer architecture can identify **important musical features such as rhythm and harmonic structure**, while being more robust to noise.

Pretrained Representation: The model was initialized from a checkpoint trained on **AudioSet**, providing strong audio feature representations before fine-tuning on the genre classification task.

- **Fine-Tuning Strategy**

Optimizer & Scheduler: Training was performed using **AdamW** with a learning rate of **5e-5** and **weight decay of 5e-4**.

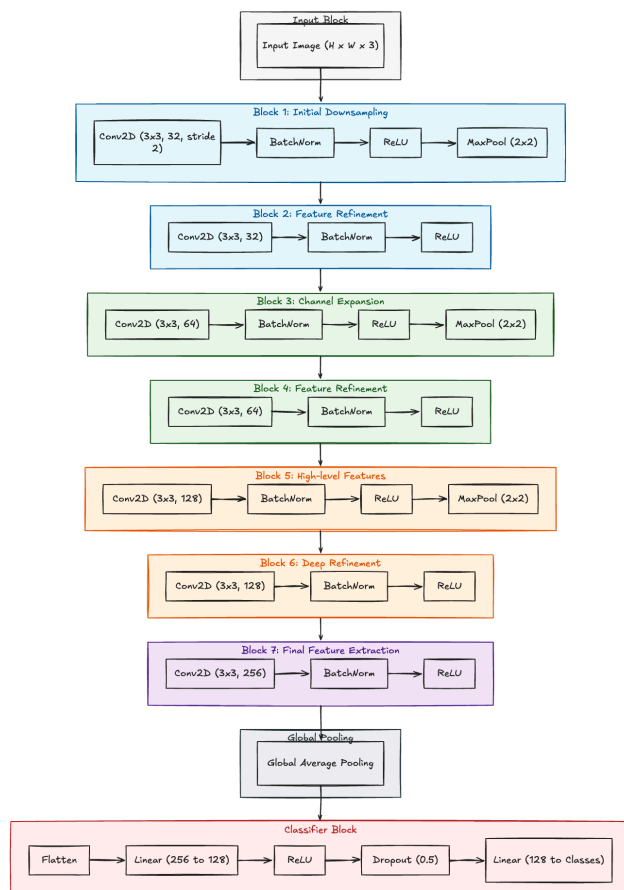
Training Duration: The model was fine-tuned for **12 epochs** on the training dataset.

Layer Freezing: Initially, the transformer backbone was frozen to train the classification head, and later the entire network was fine-tuned.

Input Adaptation: Log-Mel spectrograms were resized to match the AST input requirement of **1024 × 128**, using a **10 ms hop length** during spectrogram generation.

5.2. Model 2: Custom Deep Learning Model (Scratch CNN)

- **Architecture**



Custom model used in this project is a Convolutional Neural Network (CNN) trained from scratch to classify music genres from Mel spectrogram inputs of size $(1 \times 128 \times 512)$. The architecture consists of multiple convolutional blocks followed by batch normalization and ReLU activation to extract meaningful time–frequency patterns from the spectrograms.

The network progressively increases the number of feature channels from $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ while applying max pooling layers to reduce the spatial dimensions and capture hierarchical audio features. After the convolutional feature extraction stage, a Global Average Pooling layer reduces the feature map to a compact representation. Finally, a fully connected classifier with dropout is used to predict the final genre class among the 10 genres.

- **Salient Points:**

Mel Spectrogram Input: Audio files were converted into Mel spectrograms, allowing the CNN to treat them like images and learn time–frequency patterns of music.

Deep Convolution Blocks: Multiple convolution layers help capture hierarchical audio features such as rhythm, harmonics, and spectral textures associated with different genres.

Batch Normalization: Used after convolution layers to stabilize training and speed up convergence.

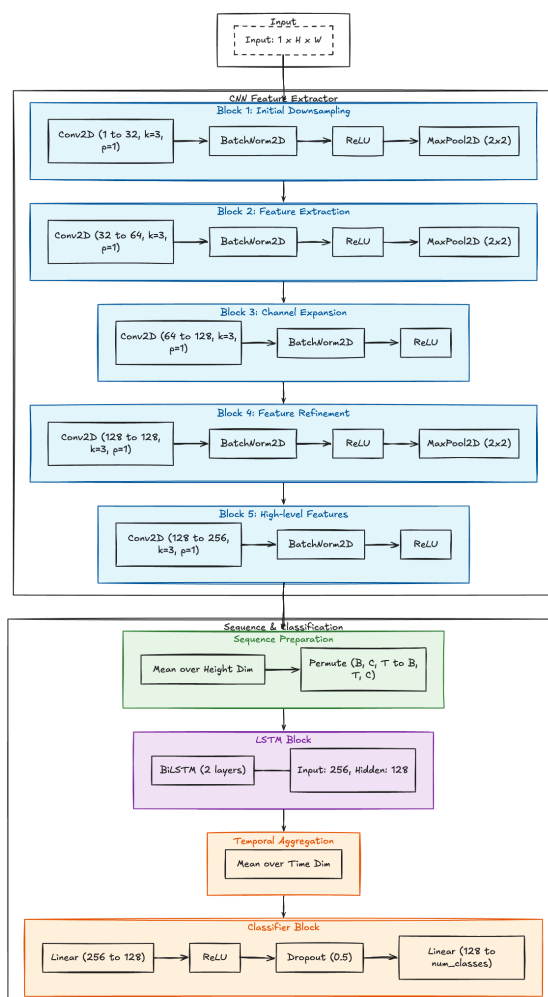
Pooling Layers: Max pooling reduces spatial dimensions and helps the model focus on the most important features.

Global Average Pooling: Reduces the number of parameters and helps prevent overfitting by summarizing feature maps.

Dropout Regularization: A dropout layer (0.5) improves generalization and reduces overfitting during training.

5.3. Model 3: RNN-Based Model (CNN-LSTM)

- **Architecture:**



This model combines **Convolutional Neural Networks (CNN)** and **Long Short-Term Memory (LSTM)** to capture both spatial and temporal patterns in the audio spectrograms. The input Mel spectrogram of size $(1 \times 128 \times 512)$ is first processed through several convolutional layers that extract time–frequency features. These CNN layers increase feature channels from $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ while reducing spatial dimensions using max pooling.

After the CNN feature extraction stage, the feature maps are reshaped into a **sequence format**, where the time dimension is treated as a sequence for the recurrent network. A **2-layer Bidirectional LSTM** is then used to model temporal dependencies in the audio signal. Finally, the output from the LSTM is passed through fully connected layers with

dropout to produce the final **genre classification across 10 classes**.

- **Salient Points:**

Combination of CNN and RNN: CNN extracts spatial time-frequency features from spectrograms, while LSTM models the temporal progression of audio signals.

Temporal Modeling: Music signals contain sequential patterns such as rhythm and beat structure, which are effectively captured using an LSTM.

Bidirectional LSTM: Allows the model to learn context from both past and future time steps in the audio sequence.

Feature Compression: Frequency dimensions are averaged before feeding into the LSTM to convert spectrogram features into a manageable sequence.

Dropout Regularization: Helps reduce overfitting and improves generalization during training.

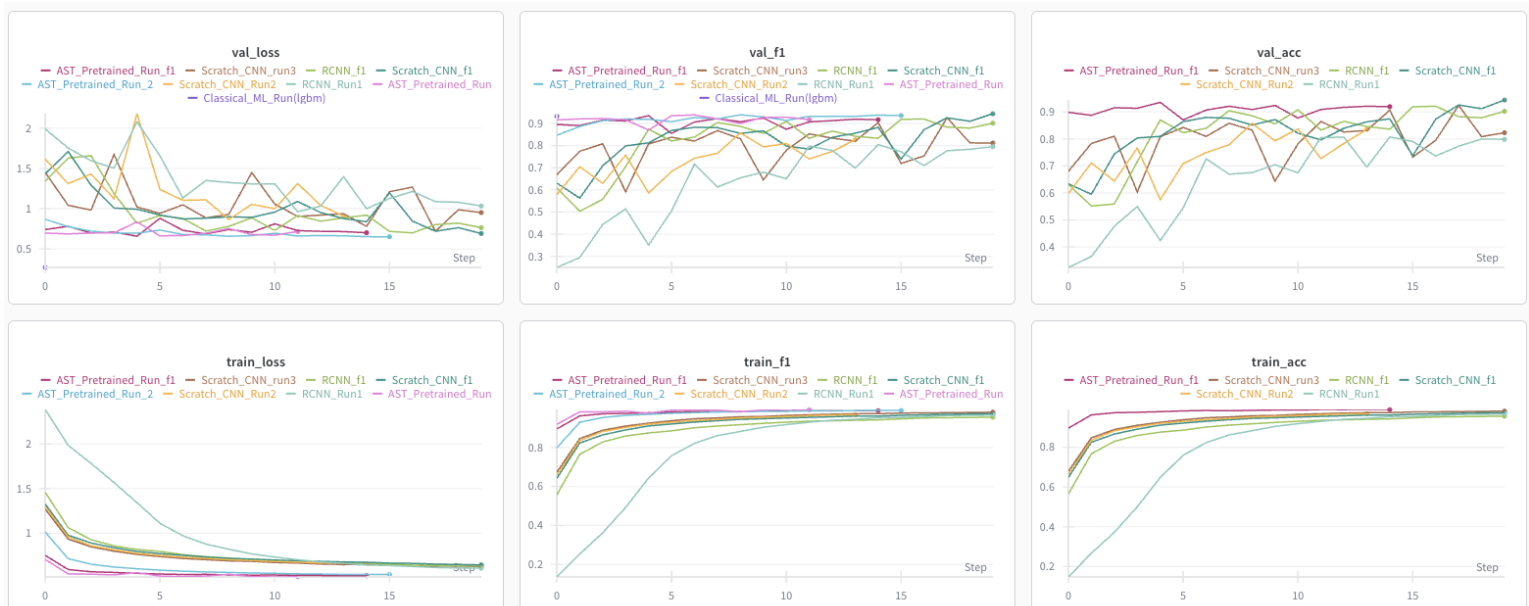
6. Performance & Comparative Analysis

Evaluation Metrics

The primary evaluation metric used in the Kaggle competition was the **Macro F1 Score** across the 10 genre classes. This metric evaluates model performance equally across all classes and is suitable for multi-class classification tasks where balanced performance is important. During training, the **Macro F1 Score** was calculated and logged using Weights & Biases (wandb) to monitor model performance. In addition, **accuracy** was also computed for both training and validation datasets to track overall classification performance.

Training Performance

The training process for each model was monitored using wandb, where metrics such as **training loss**, **validation loss**, **F1 score**, and **accuracy** were logged across epochs. The training curves show that all models improved steadily during training, with loss decreasing and F1 score increasing over time. The **Scratch CNN** model showed the most stable convergence and highest validation performance. The **AST** model also performed well, demonstrating strong learning capability for spectrogram features. The **RCNN** model converged more slowly and achieved comparatively lower performance.



Comparative Results:Analysis:

Model	Validation F1	Accuracy	Kaggle Macro F1	Observations
Scratch CNN	~0.94	~0.92	0.948	Best performance and stable training
AST (Audio Spectrogram Transformer)	~0.90–0.93	~0.90	0.93	Captures global audio patterns well
RCNN	~0.80	~0.82	0.802	Slower convergence and lower performance

From the experiments, the **Scratch CNN architecture achieved the best performance**, likely because convolutional layers effectively capture **local time–frequency patterns in Mel spectrograms**. The **AST model**, which uses a transformer-based architecture, performed competitively but required more computational resources and training time. The **RCNN model** combined convolutional and recurrent layers but did not outperform the simpler CNN architecture for this task.

Kaggle Performance

The final submission using the **Scratch CNN model achieved a Macro F1 Score of 0.948 on the Kaggle leaderboard**, which was the best-performing model among the tested architectures. This result shows that the preprocessing pipeline, augmentation techniques, and model selection were effective for handling noisy mashup audio and performing accurate genre classification.

7. Conclusion & Future Work

- **Key Learnings:** This project provided valuable experience in applying deep learning techniques to a challenging audio classification task. Initially, I had no prior experience working with audio datasets, which made understanding audio processing and spectrogram representations difficult in the beginning. Over time, I developed a strong understanding of audio preprocessing, Mel spectrogram generation, and training deep learning models for audio tasks.
- **Challenges Faced:** One of the main challenges was bridging the gap between **clean training data and noisy mashup test samples**. This was addressed by introducing **noise augmentation, time masking, and frequency masking**, which helped the

models generalize better. Through experimentation with different architectures such as **Scratch CNN, RCNN, and AST**, I learned how model design and data preprocessing significantly impact performance.

Areas for Improvement: For future work, I would focus more on **improving the training dataset and augmentation strategies**, as the quality and diversity of training data play a crucial role in achieving better results. Exploring **more advanced augmentation techniques, ensemble models, and further hyperparameter tuning** could potentially lead to improved performance on similar audio classification tasks.

8. Rerences

1. **Valerio Velardo – The Sound of AI**, Audio Signal Processing for Machine Learning, YouTube Playlist.
Link :[click here](#)
2. **Valerio Velardo – The Sound of AI**, PyTorch for Audio + Music Processing: Course Overview, YouTube Playlist.
Link : [click here](#)
3. **Y. Gong, Y. Chung, and J. Glass**, AST: Audio Spectrogram Transformer for Audio Classification, Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2021.
Available: [click here](#)

9. Deployment Link : [Click here](#)